

第7章 文件与数据交换

pathlib · 编码与 open · 流式处理 · JSON / CSV · 标准输入输出

本章目标:讲透 Python 文件 I/O 的完整图景;pathlib 路径对象、open 的模式与编码、按行流式处理大文件、二进制与 struct 打包、JSON/CSV 结构化交换,以及 sys.argv/stdin/stdout 命令行管线。学完你能安全、高效、编码正确地处理生产中的数据文件。

直觉起点:C 里文件是 FILE* 加一串函数;Python 里文件是对象,路径也是对象。C 的核心痛苦——路径拼接、编码、缓冲区——在 Python 里都有语法级解药,但"模式选错清空文件"、"编码没指定乱码"这类坑依然存在,只是换了形态。

前置要求:第1章字符串与字节、第5章 with 上下文管理器(文件必须 with 打开)。

本章路线:第一阶段(1.5 小时)读 §7.1–§7.3,掌握 pathlib 与文本流式读取;第二阶段(1.5 小时)读 §7.4–§7.6,掌握二进制与 JSON/CSV;第三阶段(2 小时)读 §7.7–§7.8 并完成章末实验。

本章知识导图

- └─ 7.1 pathlib —— 路径是对象,不是字符串
- └─ 7.2 open 与编码 —— 模式、encoding、BOM、换行
- └─ 7.3 流式读大文件 —— 按行迭代,内存恒定
- └─ 7.4 二进制文件 —— struct 打包与解包
- └─ 7.5 JSON —— 序列化、反序列化与中文
- └─ 7.6 CSV —— DictReader、写行、编码陷阱
- └─ 7.7 argv 与标准流 —— 命令行管线
- └─ 7.8 生产实践 —— 临时文件与错误处理

§7.1 pathlib:路径是对象,不是字符串

C 的路径处理是 char 数组拼接:strcpy、strcat、检查末尾斜杠,一不留神缓冲区溢出或平台分隔符错误。Python 3.4+ 的 **pathlib** 把路径变成对象,拼接、判断、遍历都是方法调用,跨平台且不可变:

```
from pathlib import Path

base = Path("data") / "2026" / "logs"      # 运算符拼接,自动处理分隔符
print(base)                                # data\2026\logs (Windows)

p = Path("data/2026/logs/app.log")
print(p.name)          # app.log
print(p.stem)          # app
print(p.suffix)        # .log
print(p.parent)        # data\2026\logs
print(p.exists())      # False — 当前目录没有这个文件

for child in Path("data").iterdir():        # 遍历目录
    print(child.name)
```

C(字符串拼接)	Python(pathlib)
<code>strcat(buf, "/"); strcat(buf, name);</code>	<code>Path("data") / name</code>
平台分隔符要手动处理 / 转义	Path 自动使用当前平台分隔符
判存在 <code>stat() + errno</code> 组合拳	<code>p.exists()</code> / <code>p.is_dir()</code> / <code>p.is_file()</code>
目录遍历要 <code>opendir/readdir/closedir</code>	<code>p.iterdir()</code> / <code>p.glob("*.log")</code>
拼错路径只能靠运行期报错	Path 对象先行验证合法性,错误更早

工程建议 1:新代码一律用 `pathlib`,不用 `os.path` 拼接字符串;函数参数类型标注 `path: Path`,同时用 `pathlib.Path` 作为唯一入口,避免"字符串和 Path 混用"的隐式转换错误。

7.1.1 常用操作

读文件: `Path.read_text(encoding=...)`、`read_bytes()`; 写文件: `write_text()`、`write_bytes()`; 建目录: `makedirs(parents=True, exist_ok=True)`; 通配: `glob("**/*.log")` 递归找日志。注意 `read_text/write_text` 是"一把梭"API,适合小文件;大文件仍用 `open` 流式处理。

Windows 下拼路径还有一坑:字符串里反斜杠是转义符, `"C:\data"` 必须写成双反斜杠;换成 Path 之后这类问题彻底消失。迁移期混用 `os.path` 与 `pathlib` 会让类型标注失真,团队约定:函数签名一律 Path,内部统一 Path 方法。

§7.2 open 与编码:模式、encoding、换行

`open` 的完整调用是 `open(path, mode, encoding=...)`。模式的首字符决定核心行为:

```
with open("a.txt", "w", encoding="utf-8") as f:
    f.write("第一行\n")          # "w" 覆盖写,文件不存在则创建

with open("a.txt", "a", encoding="utf-8") as f:
    f.write("追加行\n")          # "a" 追加写,不覆盖已有内容

with open("a.txt", "r", encoding="utf-8") as f:
    print(f.read())              # "r" 默认模式,读取全部文本
```

模式	含义	C 对应
"r"	读文本(默认),文件必须存在	"r"
"w"	写文本,立即清空原内容	"w"
"a"	追加写,不清空,从末尾开始	"a"
"x"	独占新建,文件已存在则报错	无(要手写 O_EXCL)
"r+"	读写,不清空,光标在开头	"r+"
"rb" / "wb"	二进制读/写,不做编码解码	"rb" / "wb"

写文件还有一个"看不见的延迟":内容先落在用户态缓冲区,进程崩溃时未 `flush` 的数据直接丢失。with 退出或显式 `f.flush()` 才落盘;日志系统常用 `flush=True` 保证每条即时可见,数据库类写入则靠 `fsync` 兜底。理解"缓冲区存在"这个事实,排障时少走弯路。

误区:不指定 encoding。 Windows 默认用 GBK 打开文本文件,Linux 默认 UTF-8——同一份代码两平台行为不同。
生产规范:所有文本 open 一律显式 `encoding="utf-8"`,写文件同理,否则读 UTF-8 文件在 Windows 上直接 `UnicodeDecodeError`。

7.2.1 BOM 与换行

Windows 记事本保存的 UTF-8 文件带 BOM(文件头三个字节 EF BB BF),Python 读它会多出一个不可见字符 `U+FEFF`。用 `encoding="utf-8-sig"` 读可自动剥掉 BOM;写文件想带 BOM 给别人用记事本看,也用 `utf-8-sig` 写。

```
with open("bom.txt", "rb") as f:
    head = f.read(3)          # 二进制读文件头三个字节
    print(head == b"\xef\xbb\xbf")  # True — BOM 确实存在

with open("bom.txt", encoding="utf-8-sig") as f:
    text = f.read()           # utf-8-sig 自动剥掉 BOM

print("BOM 已剥离:", "" not in text)
```

换行:open 默认开启通用换行模式,读时把 `\r\n` 统一转成 `\n`,写时按平台转换。若与别人交换的字节必须原样,用 `newline=""` 关闭转换(写 CSV 时这是标准操作)。

§7.3 流式读大文件:内存恒定

C 里处理大文件的正道是 `fgets` 逐行;Python 的对应物是直接迭代文件对象。文件对象是惰性的,一次只从磁盘读一个块,因此 10 GB 日志也能处理,内存占用与文件大小无关:

```
def count_levels(path: Path) -> dict:
    """统计日志中各级别出现次数,内存与文件大小无关"""
    stats = {"INFO": 0, "WARN": 0, "ERROR": 0}
    with open(path, encoding="utf-8") as f:
        for line in f:          # 逐行迭代,绝不 read() 全量
            for level in stats:
                if level in line:
                    stats[level] += 1
    return stats

print(count_levels(Path("data/app.log")))  # 例如 {'INFO': 42, ...}
```

C(流式)	Python(流式)
<code>while (fgets(buf, N, fp))</code>	<code>for line in f:</code>
buf 要自己开、自己管长度上限	迭代器按行切分,行长度无上限
行内再 <code>strtok</code> 拆分,小心改坏原串	<code>line.split()</code> 返回新列表,原行不动
忘记 <code>fclose</code> 泄漏句柄	<code>with</code> 退出自动 close

迭代文件对象还有两个实用细节:空行也会被迭代出来,需要时自行 `strip` 判断;行尾换行符保留着,做精确匹配先 `line.rstrip()`。若按"固定块"而非"行"处理(如解析二进制日志),用 `f.read(chunk_size)` 循环读固定字节数,同样内存恒定。

误区:大文件一把梭。 `f.read()` 把整个文件读进内存,100 MB 文件就是 100 MB 内存;若文件再大,进程直接 OOM。凡是"逐行处理"的场景都用迭代;只有文件确定很小(配置、模板)才用 `read()`。

§7.4 二进制文件:struct 打包与解包

C 用 `fwrite/fread` 加结构体直接怼文件,但内存布局依赖编译器与平台;Python 用 `struct` 模块按格式串精确控制字节序与字段宽度,跨平台可移植:

```
import struct

# 格式串 ">iif":大端(int32, int32, float32)
data = struct.pack(">iif", 7, 100, 3.14)
print(len(data))                # 12 字节

with open("bin.dat", "wb") as f:
    f.write(data)

with open("bin.dat", "rb") as f:
    raw = f.read()               # 小文件,一次读完没问题
    a, b, c = struct.unpack(">iif", raw) # (7, 100, 3.1400001049...)
    print(a, b, round(c, 2))
```

C(fwrite/fread)	Python(struct)
结构体直接写,布局依赖 ABI	格式串显式声明字节序与宽度
跨机器/跨编译器读不回原样	> 大端 / < 小端由你指定,结果可移植
填充字节(padding)不可控	= 与 > 控制是否对齐,字段宽度精确
读回要逐字段 memcpy 校验	unpack 一次解出,长度不匹配直接抛错

生产场景:与 C 服务通信的二进制协议、图片头解析、定长记录文件。记住格式串语法:整数用 `i`(4 字节)/`q`(8 字节),浮点用 `f`(4)/`d`(8),字符串用 `s` 前缀长度,如 `"10s"` 表示定长 10 字节字符串。

`struct` 格式串可以组合出复杂协议,比如 `">H2sH"` 表示无符号短整型、两字节字符串、无符号短整型共 6 字节。解析外部协议时先验证 `len(raw) == struct.calcsize(fmt)`,不匹配立刻报错而不是盲取——这正是 C 里缓冲区越界的前身,在 Python 里变成显式的长度检查。

§7.5 JSON:生产数据交换第一格式

JSON 是服务间交换数据的默认格式。Python 的 `json` 模块四个函数覆盖全部需求:`loads`(字符串→对象)、`dumps`(对象→字符串)、`load`(文件→对象)、`dump`(对象→文件):

```

import json
from pathlib import Path

order = {
    "order_id": "A1001",
    "customer": "李雷",
    "items": [{"sku": "P-001", "qty": 2}],
    "paid": True,
}

text = json.dumps(order, ensure_ascii=False, indent=2)
Path("order.json").write_text(text, encoding="utf-8")

loaded = json.loads(Path("order.json").read_text(encoding="utf-8"))
print(loaded["customer"])          # 李雷
print(loaded["items"][0]["qty"])   # 2

```

误区:中文变成 \uXXXX。 dumps 默认 ensure_ascii=True,所有非 ASCII 字符转成 \u 转义,中文文件变成天书——功能没错,但难读、体积大、diff 混乱。生产规范:写给人读或与业务方对齐的文件,一律 ensure_ascii=False 且文件编码 UTF-8。

类型映射要心里有数:JSON 的 true/false/null 对应 Python 的 True/False/None;数字只有一种,Python 的 int 读回来可能是 int 或 float。Python 的 set、datetime 不能直接序列化,需要自定义 default 函数转换——第10章的类型体系会再讲。

一个生产级细节:json.load 读入的 dict,访问不存在的 key 会抛 KeyError。对第三方接口的返回,先判断字段是否存在再取值,或用 dict.get(key, default)。同理,dumps 前确认数据里没有 set、bytes 这类不可序列化对象,否则运行到一半才抛 TypeError,殃及整批数据落盘。

§7.6 CSV:表格数据的最简载体

csv 模块处理带引号、带逗号、带换行的字段,比自己 split(",") 可靠得多。生产首选 DictReader——用表头当 key,代码可读且不怕列顺序变化:

```

import csv
from pathlib import Path

rows = [
    {"name": "李雷", "dept": "研发", "salary": 18000},
    {"name": "韩梅梅", "dept": "销售", "salary": 12000},
]

with open("staff.csv", "w", encoding="utf-8-sig",
        newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "dept", "salary"])
    writer.writeheader()
    writer.writerows(rows)

with open("staff.csv", encoding="utf-8-sig", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["dept"])    # 按表头取值

```

两个必知细节: `newline=""` 关闭换行转换,防止 Excel 打开出现空行——这是 csv 写操作的标准配置;读别人给的 CSV 先确认编码(utf-8 无 BOM / 带 BOM / GBK),BOM 用 utf-8-sig 自动剥离。Excel 生成的 CSV 常是 GBK,Windows 上读这类文件要 `encoding="gbk"`。

还有一类常见需求:表头缺失或列数不齐的脏 CSV。DictReader 遇到缺字段的行会得到缺 key 的 dict,取值时用 `row.get("col")` 配合默认值;列顺序则完全不敏感。若 CSV 来自外部且内容不可信,可先 `csv.Sniffer` 嗅探分隔符与引号风格,再决定参数。

§7.7 argv 与标准流:命令行管线

Unix 哲学"一个程序做好一件事,输出接给下一个程序"。Python 程序通过 `sys.argv` 拿参数、`sys.stdin` 读输入、`sys.stdout` 写输出,天然参与管道:

```
import sys

def main() -> None:
    if len(sys.argv) < 2:
        print("用法: python word_count.py 文件名", file=sys.stderr)
        sys.exit(1)
    path = sys.argv[1]
    total = 0
    with open(path, encoding="utf-8") as f:
        for line in f:
            total += len(line.split())
    print(total) # 标准输出,可被管道接走

if __name__ == "__main__":
    main()
```

C	Python
<code>int main(int argc, char *argv[])</code>	<code>sys.argv[0]</code> 是脚本名, <code>sys.argv[1:]</code> 是参数
参数个数靠 <code>argc</code> 判断	<code>len(sys.argv)</code> 判断
<code>printf</code> 打印,错误码 <code>return</code>	<code>print</code> 打印, <code>sys.exit(1)</code> 带状态码退出
<code>getchar/gets</code> 读 <code>stdin</code>	<code>for line in sys.stdin:</code> 直接迭代
错误输出 <code>stderr</code> 手动 <code>flush</code>	<code>print(..., file=sys.stderr)</code>

参数多了就上 `argparse` 标准库:自动生成帮助、类型转换、必选可选参数,比手写 `sys.argv` 解析稳健得多。第13章讲项目工程化时会用到。

`stdout` 的编码同样值得注意:Windows 控制台默认 GBK,print 中文一般没问题,但输出重定向到文件时按控制台编码写入,可能乱码或报错。生产脚本开头 `sys.stdout.reconfigure(encoding="utf-8")` 统一输出编码,是跨平台排障的常规动作。

§7.8 生产实践:临时文件与错误处理

写文件最容易半途而废:磁盘满、进程被杀,目标文件只剩一半。两条生产铁律:**原子替换**——先写临时文件,成功后 `rename` 覆盖;以及**用 `tempfile` 生成临时文件**,不自己拼路径:

```

import os
import tempfile
from pathlib import Path

def atomic_write(path: Path, content: str) -> None:
    """先写同目录临时文件,再原子替换,避免写一半的坏文件"""
    fd, tmp_name = tempfile.mkstemp(
        dir=path.parent, prefix=path.name + ".", suffix=".tmp")
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            f.write(content)
        os.replace(tmp_name, path) # 原子操作:要么全有,要么全无
    except BaseException:
        os.unlink(tmp_name) # 出错清理,不留垃圾文件
        raise

atomic_write(Path("config.json"), '{"env": "prod"}')

```

工程建议 2:所有文件读写都放进 with;能预估会失败的解析(JSON/编码/struct 长度)用 try/except 包住并给出带文件名的错误信息: `except OSError as e: raise RuntimeError(f"读取 {path} 失败") from e` ——from 保留原始异常链,排障时能看到根因。

误区:直接覆盖用户文件,出错即毁灭。w 模式打开瞬间就把原内容清空,如果写入中途抛异常,原文件已不可恢复。写重要配置、报表、索引文件一律走"临时文件 + os.replace"的原子替换,这是生产文件写入的基本盘。

最后回顾本章主线:路径用对象、打开必带编码、读大文件走迭代、写重要文件走原子替换、交换格式统一 JSON/CSV。文件 I/O 没有玄学,只有"模式、编码、缓冲、原子性"四个词——每个词对应一个 C 时代的经典事故,也对应一条 Python 时代的纪律。

本章速查表

场景	写法
拼接路径	<code>Path("data") / "logs" / f.name</code>
读小文件	<code>Path(p).read_text(encoding="utf-8")</code>
写小文件	<code>Path(p).write_text(s, encoding="utf-8")</code>
流式逐行	<code>for line in open(p, encoding="utf-8"):</code>
追加写	<code>open(p, "a", encoding="utf-8")</code>
独占新建	<code>open(p, "x", ...)</code> ,已存在则报错
剥 BOM	<code>encoding="utf-8-sig"</code>
二进制打包	<code>struct.pack(">iif", 7, 100, 3.14)</code>
JSON 序列化	<code>json.dumps(obj, ensure_ascii=False, indent=2)</code>
CSV 读	<code>csv.DictReader(f)</code> ,取行用 <code>row["表头"]</code>
CSV 写	<code>csv.DictWriter + newline=""</code> + 统一编码
命令行参数	<code>sys.argv[1:]</code> ,参数多上 <code>argparse</code>
错误输出	<code>print(msg, file=sys.stderr)</code>
原子写文件	<code>tempfile + os.replace(tmp, target)</code>
递归找文件	<code>Path(root).glob("**/*.log")</code>
建目录	<code>p.mkdir(parents=True, exist_ok=True)</code>

最小必会清单

1. 能解释 w/a/x/r 四种模式的区别,并说出 w 的危险性与原子替换方案
2. 能说出为何所有文本 open 都要显式指定 encoding,以及 utf-8-sig 的用途
3. 能用 `for line in f` 流式统计 10 GB 文件,并解释为什么内存不涨
4. 能用 struct 打包/解包定长二进制记录,并理解字节序前缀的含义
5. 能用 json 读写中文文件(`ensure_ascii=False`)与 DictReader 处理带表头 CSV
6. 能写一个读 sys.argv、向 stdout 输出、错误走 stderr 的命令行工具

自测题

Q

Q1. `open("a.txt", "w")` 打开一个已有重要数据的文件会发生什么?正确的做法是什么?

答案:文件内容在打开瞬间被清空,后续写入失败数据也不可恢复。正确做法:先写同目录临时文件(`mkstemp`),全部成功后再 `os.replace` 原子替换;失败时删除临时文件。若只想追加内容,用 "a" 模式。

Q

Q2. 在 Windows 上执行 `open("f.txt").read()` 处理 UTF-8 文件,常报 `UnicodeDecodeError`,为什么?怎么修?

答案:未指定 encoding 时 Windows 默认用区域语言编码(GBK)解码,UTF-8 的中文字符序列在 GBK 下不合法。修复:显式 `encoding="utf-8"`;若文件带 BOM 用 `"utf-8-sig"`。生产代码所有文本 `open` 一律显式声明编码。

Q

Q3. 为什么说 `for line in f` 处理大文件内存恒定?如果写成 `f.read().splitlines()` 会怎样?

答案:文件对象是惰性迭代器,每次只从磁盘读一块并切出下一行,行处理完即释放,峰值内存与文件总大小无关。`f.read()` 一次性把全部内容装入内存,`splitlines()` 再复制一份列表,10 GB 文件直接 OOM。

Q

Q4. `struct.pack(">ii", 1, 2)` 与 `struct.pack("ii", 1, 2)` 的字节差异是什么?生产中选择哪种?

答案:前者显式大端,输出 `00 00 00 01 00 00 00 02`;后者不指定字节序,默认用本机字节序(小端机器上是 `01 00 00 00 02 00 00 00`)。生产与外部系统交换二进制一律显式指定字节序("`>`" 大端或 "`<`" 小端),避免跨平台解析错乱。

Q

Q5. `json.dumps(d)` 输出里中文全是 `\uXXXX`,影响排障与 diff,如何解决?还有哪些类型不能直接序列化?

答案:加 `ensure_ascii=False`,中文原样输出,文件再以 UTF-8 编码保存。不能直接序列化的类型:`set`、`datetime`、`bytes`、自定义类实例;解决方式是 `dumps` 的 `default` 参数提供转换函数,或先把数据转成可序列化的 `dict/list`。

Q

Q6. 用 `open("staff.csv", "w", encoding="utf-8", newline="")` 写 CSV 后,为什么必须带 `newline=""`?

答案:不带 `newline=""` 时,通用换行模式会把 `\n` 转成平台换行(Windows 为 `\r\n`),而 `csv` 模块内部已为字段内的换行加了引号与 `\r\n` 处理,双重转换导致 Excel 打开出现空行或错行。显式 `newline=""` 关闭转换,让 `csv` 模块全权负责换行。

章末实验题:日志解析与统计

实验 7.1 日志解析与统计工具(覆盖:\$7.1–\$7.7)

需求:① 接受命令行参数:输入日志路径(必选)、输出报告路径(可选,默认 `stdout`);② 按行流式解析形如 `[2026-08-01 10:00:03] [INFO] user=alice op=login` 的日志;③ 统计:各级别条数、活跃用户数、总行数、按小时分桶统计 `ERROR`;④ 结果输出为 CSV(表头 `time,level,count`)与一段摘要文本;⑤ 路径不存在或格式损坏时给出带文件名的清晰错误并返回非零退出码;⑥ 覆盖知识点自检清单写入报告文件(可选扩展)。

覆盖知识点:`pathlib` 路径处理(\$7.1)/`open` 模式与编码(\$7.2)/流式逐行读(\$7.3)/`csv` 输出与编码(\$7.6)/`sys.argv` 与 `stderr`(\$7.7)/异常与原子写(\$7.8)。

实验环境:Python 3.10+,单文件 `logstat.py`。

```

# logstat.py — 日志解析与统计(覆盖第7章全部知识点)
import csv
import re
import sys

from collections import Counter, defaultdict
from pathlib import Path

LINE_RE = re.compile(
    r"^\[(?P<ts>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]"
    r" \[(?P<level>\w+)\] user=(?P<user>\w+)"

def parse_line(line: str) -> dict | None:
    """解析一行日志;格式不对返回 None,不抛异常"""
    m = LINE_RE.match(line.strip())
    if not m:
        return None
    ts = m.group("ts")
    return {
        "hour": ts[11:13],          # 取 "10" 作为小时桶
        "level": m.group("level"),
        "user": m.group("user"),
    }

def main() -> None:
    if len(sys.argv) < 2:
        print("用法: python logstat.py 日志文件 [报告.csv]",
              file=sys.stderr)
        sys.exit(1)
    src = Path(sys.argv[1])
    if not src.is_file():
        print(f"错误: 文件不存在 {src}", file=sys.stderr)
        sys.exit(2)

    levels = Counter()
    hours = defaultdict(int)
    users = set()
    total = 0
    with open(src, encoding="utf-8") as f:
        for line in f:
            total += 1
            rec = parse_line(line)
            if rec is None:
                continue
            levels[rec["level"]] += 1
            hours[rec["hour"]] += 1
            users.add(rec["user"])

    print(f"总行数: {total} 活跃用户: {len(users)}")
    for level, n in levels.most_common():
        print(f" {level}: {n}")
    print("ERROR 高峰时段:", hours.most_common(3))

```

```
if len(sys.argv) > 2:
    write_report(Path(sys.argv[2]), hours)
```

第二段:报告输出函数。main 只负责统计,输出职责交给独立函数,便于单独测试。

```
def write_report(out: Path, hours: dict) -> None:
    """把小时桶统计写成 CSV;utf-8-sig 保证 Excel 打开不乱码"""
    with open(out, "w", encoding="utf-8-sig",
              newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["hour", "count"])
        for hour, n in sorted(hours.items()):
            writer.writerow([hour, n])
    print(f"报告已写入 {out}")

if __name__ == "__main__":
    main()
```

测试建议:先生成样例日志: `for i in $(seq 1000); do echo "[2026-08-01 10:$(date +%M):00] [INFO] user=alice op=login"; done > sample.log`,再运行 `python logstat.py sample.log report.csv`;故意删掉一行开头的 [验证容错;把文件编码改成 GBK 验证编码错误信息是否清晰。对照检查清单:路径用 pathlib(是)、显式编码(是)、流式读(是)、CSV 输出带 `newline=""`(是)、argv 与 stderr(是)、非零退出码(是)。

下一章预告:第8章 网络通信——socket 从 C 迁移、TCP/UDP 语义、粘包与长度前缀协议、requests 与 http.server 最小 Web 服务。数据开始跨越进程与机器。